

Compiler-as-a-Library: Pitfalls and Triumphs in Embedding Array Languages

V. E. McHale

vamchale@gmail.com

Abstract. We reflect on the Apple array system, a JIT compiler for a statically typed array language offering a shared library function taking a source string and returning machine code, with the goal of making using domain-specific languages (DSLs) less cumbersome to use from foreign languages. This approach sets a standard in interactivity; we use Apple to demonstrate the more fluent workflows that are possible. Apple offers a terse language with types; we sketch the advantages with examples and then point out mistakes in our language design. Finally, Array languages naturally express parallel calculations and would seem to be a fit for single instruction, multiple data (SIMD) instructions available on modern processors; we point out the ways textbook compilation of C-like languages thwarts SIMD and sketch designs for array language compilers that deliver.

Grown old, one can't collect one's
work and fix up all that's wrong,
so I send these poems to you, as if
shown to teach a boy

Xue Tao, trans. Jeanne Larsen

1 Introduction

We present the Apple array system, a compiler offered as a shared library function:

```
void* apple_compile(char*, size_*)
```

The compilers field is mature, but focuses on pipelines targeting C-like sources; there are many paths left unexplored and indeed fruitful in the case of array languages.

2 JIT Compilation

Apple features a Just-in-time (JIT) compiler so that REPL behavior is the same as compiled code. This turns out to be welcome when debugging the compiler; one can dump the intermediate representations and assembly.

A JIT compiler turns out to offer better interactivity particularly when it is used from a foreign language, viz.

```
> sliding_mean<-jit("([((+)/x)% (:x)]\\`7)")
> run(sliding_mean,seq(0,999,1.0))
array([3., 4., 5., 6.]
```

Machine code is the lowest common denominator, requiring nothing of the host language and thus universally available¹ (and guaranteed to have the same behavior). To wit, we can run the same code in Python and Janet:

```
>>> sliding_mean=apple.jit(
... '([((+)/x)% (:x)]\\`7)')
>>> sliding_mean,np.arange(0,10,dtype=np.float64)
array([3., 4., 5., 6.]

repl:1:> (def moving-average
repl:2:(> (apple/jit ``([((+)/x)% (:x)]\\`7)`))
<jit Vec (i + 7) float → Vec i float>
repl:2:> (moving-average @[1.0 2.0 3.0 ...])
@[4 5 6 7]
```

2.1 Exporting Debuggability

One benefit of a JIT compiler is that one can dump assembly in Python, R; the language exports its debug capabilities.²

We can see that the dot product uses SIMD instructions from within a Python REPL:

```
>>> import apple
>>> print(
... apple.asm('[(+)/ ((*)` (x::Vec n float) y)]'))

apple_0:
    ldr q3, [x2], #0x10
    ldr q2, [x1], #0x10
    fmla v0.2d, v3.2d, v2.2d
    subs x3, x3, #0x2
    b.GE apple_0
```

Finding out what assembly is executed for a given expression would be less fluent in Python/NumPy, J, or BQN.

¹ Kell [8] argues that this is why C has endured: it is compiled into object files that other languages can call in to, without imposing itself on the host language.

² GDB can debug a function written in C and called in another language, producing debug output in terms of the C source code.

2.2 Compiling to Assembly

Textbook methods are well articulated, and off-the-shelf methods deliver suitable performance in many cases. Apple outperforms R (but not NumPy), even without instruction scheduling and without any heuristic for picking registers to spill. Folklore would have it that writing a compiler is a tall order, but it is a mature field and by targeting a high-level language we avoid, for instance, implementing strength reduction—all loops are implicit and there are no references, so we simply generate sensible code when iterating over an array.

However, ABIs are fickle: a compiler that mistakenly clobbers registers may not show up in tests depending on the test environment, and the order in which the tests are executed may change results.

Hui and Kromberg [7] note that APL implementations have had success writing interpreters in C; a JIT compiler is necessary to get the advantages of embedability but this avoids the tortuous ABI and platform compliance.

2.3 JIT Linking

JIT compilation is not covered in classic compilers textbooks, though Eli Bendersky gives a reference on his blog [3]. JIT linking is even more esoteric. LLVM documentation mentions such a feature but its implementation is not described [1].

2.4 Statically Determined Memory Management

Motivation The language is high-level but does not depend on a garbage collector; the compiler is able to track liveness and insert `free`s as necessary. This approach was chosen so that code could be compiled on one machine and executed on another without sending a runtime or garbage collector (garbage collectors also require care with FFI [11]).

We are also able to do simple reuse of allocations based on shape information 7.3.

Polemic In order for such analysis to be possible, the language is carefully limited; there are no references. This is justifiable in a domain-specific, embedded language—doubly linked lists are left to the host language.

This approach—paring down the language for the sake of some compilation technique—deserves attention from functional programmers and the compilers field alike. High-level languages offer more constraints, but the compilers field has not turned its attention high-level languages.

Array languages are a good playground: they are high-level and express things that are efficient on hardware.

3 Interactivity

Typed functional languages are in many ways superior to Python/NumPy. However, using them can be cumbersome.

Suppose one has written a (buggy) image processing function in Futhark, with the image being loaded, processed, and displayed in a Jupyter notebook. Then one must:

1. Navigate to the Futhark source file and fix the bug
2. Navigate to a shell and invoke the compiler on the corrected code
3. Navigate to Jupyter and reload the module

Futhark is chosen as an example because the language is superior to Python/NumPy; its compiler also offers advantages, such as the ability to run on the GPU. Accelerate has similar advantages and burdens.

Apple offers a JIT compiler and thus one need not navigate out of the REPL to fix a bug. Domain-specific languages (DSLs) that take care to make feedback available within the host language might find widespread adoption.

3.1 Conciseness

In k one can sum the first 100 numbers with

```
+ /!101  
5050
```

Seven key presses! This aspect of HCI (human-computer interactions) is shunned, but it is an advantage of APL descendants. With skilled embeddings, the interactivity of array languages far exceeds popular libraries.

Apple is both typed and laconic. We can inspect a type in the REPL with a

```
> :ty \p. (+)/([x*_.x]'p)  
Vec (i + 1) float → float
```

Such a combination of static types and terseness has not been articulated.

4 SIMD

Apple started out following Appel [2] and converting functions (e.g. `x. x*2`) into sequences of steps that read arguments from some specified temporaries and write their return values to some specified temporary. This turns out to be a poor fit when implementing SIMD.

When we map over a vector with `x. x*2`, we want to read several values into a vector register, multiply by 2, and then write the results. Compiling `x. x*2` to a single canonical assembly procedure is premature; it should be treated as an entity within the compiler that dispatches code for scalar or vector multiplication as necessary.

`x. x*2` and `x. if x 1 then sqrt x else x` are both functions in structured and functional programming but must be handled differently during compilation. Conditionals appear when computing sine, logarithm, etc. Classifying the functions that do not involve cases would make compiling to SIMD less obscure.

The discordance between functions-as-they-should-be-treated-in-a-compiler and procedure-ABI is part of what makes compiler-as-a-library architectures so tantalizing. C's pipelined compilation into object files does not compose insofar as it thwarts performance.

5 Canonical Status

Apple aspires to be canonical, so that one can write a function once and benefit in Python, R, etc. In order to be useful in such a capacity, it must have adequate performance.

5.1 Hardware Integer Sizes

The compiler only supports 64-bit integers and double-precision floats. This was done to simplify implementation, however it undermines performance. An array of indices could use 32-bit integers, and copying this array would be twice as fast; indeed BQN has 8-, 16-, 32-, and 64-bit integers and marshals between them where possible/necessary [9].

Our simplification should not be considered by compiler writers targeting array DSLs. Programmers rarely reach for 16-bit integers for sake of performance [9], so language implementations taking advantage of 16-bit integers offer something interesting.

5.2 Matrix Libraries

CPUs depend on cache to use floating-point math capabilities to their full potential. Getting competitive performance requires loop tiling [2, pp. 505-506][5, p. 95][12].

Unfortunately, the compiler-as-a-library approach here does not offer a terribly interesting take on matrix multiplication—worse, Apple is 10 times slower than NumPy on M1 because Apple fails to exploit undocumented specialized hardware for matrix multiplication accessible only via system libraries [4]—opposing the goal of generating code without collecting information from the operating system first.

6 Intermediate Representations

Apple has two intermediate representations. Early on, the project had only one, with jumps and memory read/writes. This became difficult when faced with

implementing the re-rank operator, which lifts (say) a binary function taking a scalar and a vector to operate on a vector and some axis of a matrix.

To address this, a higher-level IR was added, a C-inspired syntax with `for`-loops and notation for rank, dimension, and element access.

Consider the below, which takes the sums of the rows of a matrix and collects them into a vector:

```
> :cmm [((+)/)`{1 [1]} (x::M float)]

for(T30, T30 0, T30<T16) {
  T32 = 0
  for(T29, T29 0, T29<T15) {
    T31 = T27[T29][T30] (T18)(T17)
    T28@T32 = T31
    T32 += 1
  }
  T25 = T20.dim[0]
  X23 = T20[0]
  for(T26, T26 1, T26<T25) {
    X24 = T20[T26]
    X23 += X24
  }
  T12[T14] = X23
  T14 += 1
}
```

Compare the second IR:

```
(mjump ( (reg r_30) (reg r_16)) apple_1)

apple_0:
(movtemp r_32 (int 0))
(movtemp r_29 (int 0))
(mjump ( (reg r_29) (reg r_15)) apple_3)

apple_2:
(movtemp r_31
  @(ptr r_27+
    (asl
      (+
        (* (reg r_18) (reg r_29))
        (* (reg r_17) (reg r_30)))
      (int 3))))
(write
  (ptr r_28+(asl (reg r_32) (int 3)))
  (reg r_31))
```

```

(movtemp r_32 (+ (reg r_32) (int 1)))
(movtemp r_29 (+ (reg r_29) (int 1)))
(mjump (< (reg r_29) (reg r_15)) apple_2)

apple_3:
(movtemp r_25 @(ptr r_20+(int 8)))
(movf f_23 f@(ptr r_20+(int 16)))
(movtemp r_26 (int 1))
(mjump ( (reg r_26) (reg r_25)) apple_5)

apple_4:
(movf f_24
  f@(ptr r_20+
    (+ (asl (reg r_26) (int 3)) (int 16))))
(movf f_23 (+ (freg f_23) (freg f_24)))
(movtemp r_26 (+ (reg r_26) (int 1)))
(mjump (< (reg r_26) (reg r_25)) apple_4)

apple_5:
(write
  (ptr r_12+
    (+
      (asl (reg r_14) (int 3)) (int 16)))
    (freg f_23))
(movtemp r_14 (+ (reg r_14) (int 1)))
(movtemp r_30 (+ (reg r_30) (int 1)))
(mjump (< (reg r_30) (reg r_16)) apple_0)

apple_1:
(spop (reg r_22))
(movtemp r_ret (reg r_12))

```

Array language compilers will likely need a higher-level IR.

7 Static Types

Counterintuitively, static typing encompasses dynamic typing [6]. It is not just that types prevent bugs (APL descendants more or less manage in their own ways), the inferred types can be used to generate documentation as they specify an interface [10]. Type inference with documentation generation is a language implementation feature and dynamic languages lack this feature.

When working with arrays, shape types turn out to especially good. The expression below augments a matrix by adding the supplied vector as a new column on the right.

```
> :ty \xs.\ys. (|>){0,1 [2]} (xs::Vec n float) ys
```

```

Vec i float
  → Arr (i × j) float
  → Arr (i × j + 1) float

```

The abstract `i, j` give more clarity than an example (and do not require the programmer to conjure such an example, making for quicker interactions). In the above, the programmer gives a putative constraint (`xs::Vec n float`) and the computer responds with the interface. This augments the interactivity that makes terse APL descendants so special.

7.1 Typed Rank

Apple's rank operator was designed with typing in mind; one has

```

> :ty \f.\x.\y. f`{0,1} x y
(a → Vec i b → c)
  → Arr sh a
  → Arr (i `Cons` sh) b
  → Arr sh c

```

This iterates selecting cells and arranges them into an array with the remaining axes as leading dimensions of the results. In practice, one frequently needs to transpose the results of such operations—it would be preferable to construct the output so that dimensions matched the argument.

7.2 Random Arrays

Apple provides `rand. :: Arr sh float` which fills an array with random values. Its shape must be known in order to return a value; the programmer gives a type signature.

```

> rand. 0 1
rand. 0 1 : Arr sh a
> rand. 0 1::Arr (2×3) float
Arr (2×3) [ [0.39646477, 0.84048536, 0.35333609]
            , [0.44658343, 0.31869277, 0.88642843] ]

```

This is not an imposition—the dimensions specified in the type signature would be specified as arguments in other languages.

Suppose we wish to sample from a normal distribution. To do so, we compute

$$\frac{\cos(2\pi x)}{\sqrt{-2 \log(y)}}$$

with x, y sampled from $[0, 1)$.

In Apple, we can fill a vector with 100 samples from a normal distribution like so:

```
[cos.(2* *x)*sqrt(_2*_y)]`
  (rand. 0 1::Vec 100 float)
  (rand. 0 1)
```

The second `rand. 0 1` is inferred to have the same shape because `` (zip)` requires its arguments have the same leading dimension.

Static types, even when the programmer must supply type signatures, are not *a priori* burdensome.

7.3 Shape Information During Compilation

Consider

```
xs. [((+)/x)%(:x)]\`7 xs
```

which computes the seven-day moving average. Since we know that the inner loop will be executed seven times, we know it will be executed at least once.

```
> :asm xs. [((+)/x)%(:x)]\`7 xs
```

```
apple2:
    add x7, x4, #0x2
    ldr d4, [x6, x7, LSL #3]
    fadd d2, d2, d4
    add x4, x4, #0x1
    cmp x4, x3
    b.LT apple2
    fmul d2, d2, d3
    add x3, x2, #0x2
    str d2, [x0, x3, LSL #3]
    add x2, x2, #0x1
    cmp x2, x1
    b.LT apple0
apple1:
    add sp, sp, #0x60
    ret
```

Compilers are already capable of such optimization, but we get this for free from the type inference phase; it is the same information in the backend that is exposed to the programmer.

SIMD Shape types allow the programmer to express that a vector will have an even number of elements, for instance. SIMD code

The default assembly generated to map over a vector, multiplying by 2:

```
> :asm ((*2.0)')
```

```
    tbz x3, #0, apple_2
    ldr d3, [x2], #0x8
    movz x4, #0x4000, LSL #48
    fmov d2, x4
    fmul d2, d3, d2
    str d2, [x1], #0x8
    sub x3, x3, #0x1
apple_2:
    movz x4, #0x4000, LSL #48
    dup v2.2d, x4
apple_0:
    ldr q3, [x2], #0x10
    fmul v3.2d, v3.2d, v2.2d
    str q3, [x1], #0x10
    subs x3, x3, #0x2
    b.GT apple_0
apple_1:
    ret
```

When the programmer specifies the dimension to be even by type signature:

```
> :asm [(*2.0)'](x::Vec 2*n float)]
```

```
    cmp x3, #0x0
    b.EQ apple_1
apple_2:
    movz x4, #0x4000, LSL #48
    dup v2.2d, x4
apple_0:
    ldr q3, [x2], #0x10
    fmul v3.2d, v3.2d, v2.2d
    str q3, [x1], #0x10
    subs x3, x3, #0x2
    b.GT apple_0
apple_1:
    ret
```

This does not offer much performance (the inner part of the loop is executed more often), but it makes the generated assembly clearer. And the programmer gets guarantees in terms of the (high-level) language—this is an advantage of using types in particular. Autovectorization has a reputation for being unreliable in C++ [13]; by expressing expectations in terms the programmer already knows, the question is no longer only for compiler experts.

7.4 Implicit Rank

APL descendants implicitly pick cell sizes when lifting a function; for instance `+` in BQN sums two arrays of the same shape.

Language	Laconic	Statically Typed	Implicit Rank
BQN	×		×
Remora		×	×
Apple	×	×	

Apple’s lack of implicit rank undermines its conciseness; Apple’s `(+){0,0}` is `+` in BQN.

8 Conclusion

Compilers for array languages offer the opportunity for interesting design diverging from the established literature and bearing fruit. Care and imagination on the part of array language implementators might lead to wider adoption.

References

1. Jitlink and orc’s objectlinkinglayer. <https://llvm.org/docs/JITLink.html>
2. Appel, A.: Modern Compiler Implementation in ML. Cambridge University Press (1998)
3. Bendersky, E.: How to jit - an introduction. <https://eli.thegreenplace.net/2013/11/05/how-to-jit-an-introduction> (2013)
4. Boehm, S.: Fast multidimensional matrix multiplication on cpu from scratch. <https://siboehm.com/articles/22/Fast-MMM-on-CPU> (August 2022)
5. Drepper, U.: What every programmer should know about memory (2007)
6. Harper, R.: Dynamic languages are static languages. <https://existentialtype.wordpress.com/2011/03/19/dynamic-languages-are-static-languages/> (2011)
7. Hui, R.K.W., Kromberg, M.J.: Apl since 1978. Proc. ACM Program. Lang. 4(HOPL) (Jun 2020). <https://doi.org/10.1145/3386319>, <https://doi.org/10.1145/3386319>
8. Kell, S.: Some were meant for c: the endurance of an unmanageable language. In: Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software. p. 229–245. Onward! 2017, Association for Computing Machinery, New York, NY, USA (2017). <https://doi.org/10.1145/3133850.3133867>, <https://doi.org/10.1145/3133850.3133867>
9. Lochbaum, M.: Which data types are best for primitives? <https://mlochbaum.github.io/BQN/implementation/primitive/types.html> (2024)
10. Marlow, S.: <https://haskell-haddock.readthedocs.io/latest/intro.html>
11. Munch-Maccagnoni, G., Scherer, G.: Boxroot, fast movable gc roots for a better ffi. In: ML Family Workshop. Benoît Montagu, Ljubljana, Slovenia (Sep 2022), <https://inria.hal.science/hal-03910313>
12. Salykov, A.: Beating openblas and mkl in 150 lines of c code: A tutorial on high-performance matrix multiplication. <https://salykova.github.io/matmul-cpu> (2024)

13. Vardanian, A.: Understanding simd: Infinite complexity of trivial problems. <https://www.modular.com/blog/understanding-simd-infinite-complexity-of-trivial-problems> (2024)